

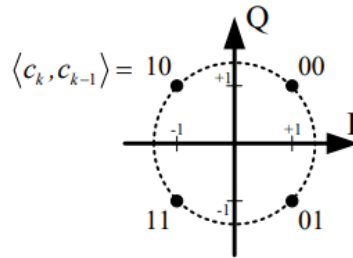
Viterbi Decode for MLSE

B1121123 巫煊崐

Consider a QPSK system, which discrete-time equivalent channel is given by $p_0 = 1$, $p_1 = 0.3$, and $p_2 = 0.7$, and which equivalent discrete-time noise is a discrete-time WSS Gaussian random process with

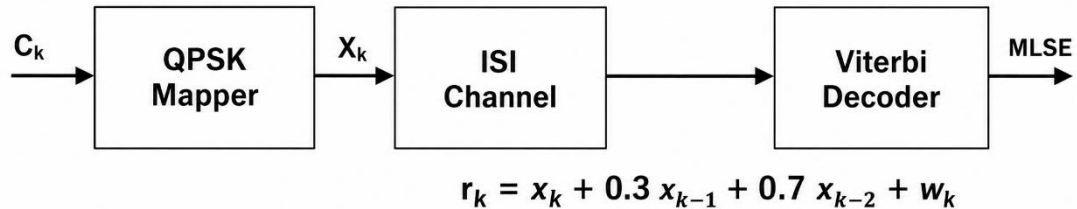
$$E[w_k w_j^*] = \begin{cases} 1, & k = j \\ 0, & \text{else} \end{cases}$$

Assuming the constellation is as shown below.



這是一個 QPSK 系統，但通道不是理想通道，有 ISI 影響

下圖為 block diagram



在第 k 個時間點，接收到的訊號不只受到目前符號 x_k 影響，也受到前面兩個符號影響，接收訊號可以寫成

$$r_k = x_k p_0 + x_{k-1} p_1 + x_{k-2} p_2 + w_k$$

其中 $p_0 p_1 p_2$ 是通道的 tap coefficients，表示不同符號對目前接收訊號的影響程度。目前符號為 $x_k p_0 = x_k$ ，前一個符號會造成 $x_{k-1} p_1 = x_{k-1} 0.3$ 的干擾，前兩個符號則會造成 $x_{k-2} p_2 = x_{k-2} 0.7$ 的干擾。

目前傳送的是 x_k ，但接收端看到的並不只是 x_k ，而是三個 symbol 的疊加：

$$x_k + 0.3 x_{k-1} + 0.7 x_{k-2}$$

例如: $x_k = 1 + j$, $x_{k-1} = -1 + j$, $x_{k-2} = 1 - j$

若不考慮雜訊，接收值為

$$r_k = (1 + j) + 0.3(-1 + j) + 0.7(1 - j) = 1.4 + 0.6j$$

題目給的雜訊功率是 1，WSS 是 wide-sense stationary，表示平均值是常數，correlation 只與時間差有關。

題目給的 QPSK，共有四個點：

Bit	Symbol
00	$1 + j$
01	$1 - j$
10	$-1 + j$
11	$-1 - j$

1. First, sketch one section of the trellis diagram for this system. You should give a clear definition of your state vector and label a few “typical” branches.

通道輸出為 $r_k = x_k + 0.3 x_{k-1} + 0.7 x_{k-2} + w_k$

所以要計算第 k 個輸出，需要知道： x_{k-1}, x_{k-2}

其中 x_k 是目前的輸入，不需要事先存進 DFF，要計算目前的輸出，必須知道前兩個符號，因此可以把狀態定義為： $S_k = (x_{k-1}, x_{k-2})$ 。

每一個符號有 4 種可能，而狀態中有兩個符號，所以狀態數為 16

初始狀態是 $S_k = (x_{-1}, x_{-2}) = (1 + j, 1 + j)$

以 y_0 為例：收到的值是 $y_0 = 0.0222 - j0.0169$ ，考慮四種可能

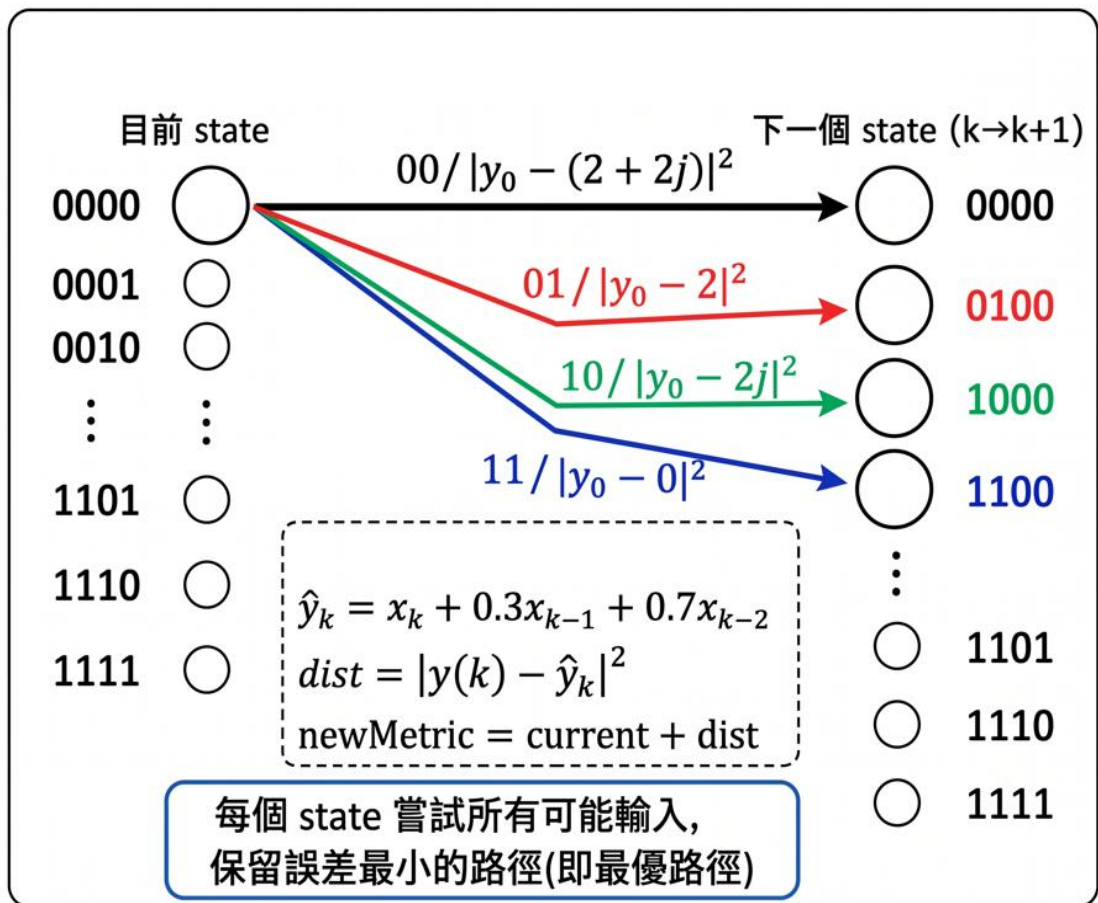
如果 $x_0 = 1 + j$

$$\begin{aligned}\hat{y}_0 &= (1 + j) + 0.3(1 + j) + 0.7(1 + j) \\ &= 2 + 2j\end{aligned}$$

Branch metric 會是 $|y_0 - (2 + 2j)|^2$ 以此類推，我們也可以用同樣方式得到其他三個，分別為

$$\begin{aligned}\text{if } x_0 = 1 - j \text{ then } &|y_0 - 2|^2 \\ \text{if } x_0 = -1 + j \text{ then } &|y_0 - 2j|^2 \\ \text{if } x_0 = -1 - j \text{ then } &|y_0 - 0|^2\end{aligned}$$

可以得到 trellis diagram



上圖為初始狀態的更新

2. Assuming that a sequence of QPSK symbols $x_{-2}, x_{-1}, x_0, \dots, x_9, x_{10}, x_{11}$, is transmitted, where x_{-2}, x_{-1}, x_{10} and x_{11} are all known to be $(1 + j)$. The corresponding sequences of symbol-spaced received signal symbols are given by

$$\begin{bmatrix} y_0 \\ \vdots \\ y_{11} \end{bmatrix} = \begin{bmatrix} 0.0222 - j0.0169 \\ 1.4509 - j0.7471 \\ -1.6027 - j1.9480 \\ 1.6105 - j2.2362 \\ -1.1898 + j0.1263 \\ -0.6067 + j0.8870 \\ 0.0579 - j0.1223 \\ 0.6309 - j0.4483 \\ -0.0330 + j0.2217 \\ 1.5283 - j1.6817 \\ 0.4960 + j1.1453 \\ 2.3859 + j0.7010 \end{bmatrix}$$

3. Using MATLAB to find the transmitted symbols $x_0, \dots, x_9$ based on the maximum likelihood sequence estimator. Please show your program and simulation result in your report and final presentation.

這題是 QPSK 通過 ISI 的通道後，使用 Viterbi 演算法找出最可能的傳送符號序列。題目給的 cost function 是 maximum likelihood，所以 path metric 是

$$J = \sum_{k=0}^{11} |y_k - \hat{y}_k|^2$$

而 branch metric 是

$$j_k = |y_k - \hat{y}_k|^2$$

把一路經過的 branch metric 加起來，就是 path metric

演算法

① 初始化

只考慮初始狀態 $x_{-1} = x_{-2} = 00 \Rightarrow (a, b) = (0, 0) \Rightarrow \text{idx} = 1$ 的累積誤差為 0。
其他皆為 ∞ 。

② 主迴圈 (time = 1 ~ 12)

- 若 $\text{time} \leq 10$: input 可為 0~3 (未知)
- 若 $\text{time} > 10$: 已知 $x_{10} = x_{11} = 00 \Rightarrow \text{input} = 0$

對每個目前狀態 (a, b) :

- 嘗試所有輸入
- 計算輸出 $\hat{y}_k$ 、誤差 d 、新的累積誤差 newMetric
- 若 newMetric 比 pathMetric(time+1, newIdx) 小，
就更新 pathMetric, fromState, usedSym

③ 回溯 (time = 12 $\rightarrow$ 1)

從最終狀態 $\text{idx} = 1$ (因為最後兩個符號已知為 00，所以狀態應回到 (0,0))
一路往回追溯，取出每一步選擇的輸入符號 (0~3)。

④ 輸出

將 symbolIndex(1~10) 轉成星座點輸出，並印出最小累積誤差。

MATLAB Simulation

```
clear; clc;
```

```
symbol = [
```

```
    1+1i
```

```
    1-1i
```

```
   -1+1i
```

```
   -1-1i
```

```
];
```

```
y = [
```

```
    0.0222 - 1i*0.0169
```

```
    1.4509 - 1i*0.7471
```

```
   -1.6027 - 1i*1.9480
```

```
    1.6105 - 1i*2.2362
```

```
   -1.1898 + 1i*0.1263
```

```
   -0.6067 + 1i*0.8870
```

```
    0.0579 - 1i*0.1223
```

```
    0.6309 - 1i*0.4483
```

```
   -0.0330 + 1i*0.2217
```

```
    1.5283 - 1i*1.6817
```

```

0.4960 + 1i*1.1453
2.3859 + 1i*0.7010
];

state_1 = [0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3]; % x(k-1)
state_2 = [0 1 2 3 0 1 2 3 0 1 2 3 0 1 2 3]; % x(k-2)

%% Viterbi
pathMetric = inf(16, 13); % pathMetric(idx, time+1)
fromState = zeros(16, 12); % fromState(newIdx, time)
usedSym = zeros(16, 12); % usedSym(newIdx, time)

% 初始狀態 x(-1) = x(-2) = 0
pathMetric(1, 1) = 0;

for time = 1:12

    if time <= 10
        tryInputs = 0:3;
    else
        tryInputs = 0; % x(10) = x(11) = 0
    end

    for idx = 1:16
        current = pathMetric(idx, time);

        a = state_1(idx); % x(k-1)
        b = state_2(idx); % x(k-2)

        for input = tryInputs

            expectedY = symbol(input+1) + 0.3 * symbol(a+1) + 0.7 *
symbol(b+1);
            dist = abs(y(time) - expectedY)^2;
            newMetric = current + dist;

            % (a,b) ----> (input,a)
            newIdx = input*4 + a + 1;

```

```

        if newMetric < pathMetric(newIdx, time+1)
            pathMetric(newIdx, time+1) = newMetric;
            fromState(newIdx, time) = idx;
            usedSym(newIdx, time) = input;
        end
    end
end

end

end

% Traceback
% 已知最後回到 state (0,0), 所以 idx = 1
idx = 1;
symbolIndex = zeros(1, 12);

for time = 12:-1:1

    % 第 time 步使用的 symbol
    symbolIndex(time) = usedSym(idx, time);

    % 回到上一個 state
    idx = fromState(idx, time);
end

fprintf('估計出的 x0 ~ x9:\n');
for time = 1:10
    sIdx = symbolIndex(time);
    s = symbol(sIdx+1);
    bits = dec2bin(sIdx, 2);
    fprintf('x_%d = %.0f %.0fj, bits = %s\n', time-1, real(s), imag(s),
bits);
end
fprintf('\n 最小誤差 = %.8f\n', pathMetric(1, 13));

%% Brute-force 驗證 Viterbi 輸出
best = inf;
best_seq = [];

for x0 = 0:3

```

```

for x1 = 0:3
for x2 = 0:3
for x3 = 0:3
for x4 = 0:3
for x5 = 0:3
for x6 = 0:3
for x7 = 0:3
for x8 = 0:3
for x9 = 0:3
    x = [x0 x1 x2 x3 x4 x5 x6 x7 x8 x9 0 0];

    total = 0;

    for k = 1:12
        if k-1 >= 1
            a = x(k-1);
        else
            a = 0;
        end

        if k-2 >= 1
            b = x(k-2);
        else
            b = 0;
        end

        expectedY = symbol(x(k)+1) + 0.3 * symbol(a+1) + 0.7 * symbol(b+1);
        total = total + abs(y(k) - expectedY)^2;
    end
    if total < best
        best = total;
        best_seq = [x0 x1 x2 x3 x4 x5 x6 x7 x8 x9];
    end
end
end
end
end
end
end
end

```

```

end
end
end
end
fprintf('\nbrute-force 最小誤差 = %.8f\n', best);
fprintf('brute-force 序列 = %s\n', mat2str(best_seq));

```

pathMetric

	1	2	3	4	5	6	7	8	9	10	11	12	13
1	0	7.9796	15.8276	15.9576	16.8673	8.5415	7.9810	9.1363	8.8801	8.0231	9.9646	6.3679	0.7346
2	Inf	Inf	8.8235	8.6716	10.0319	3.3882	13.1906	12.0975	3.7469	4.5360	12.4734	0.5755	Inf
3	Inf	Inf	11.6175	17.8780	11.9491	11.5957	4.3382	4.8190	13.2339	10.0671	5.6266	11.1163	Inf
4	Inf	Inf	4.6134	10.5920	5.1137	6.4425	9.5478	7.7802	8.1007	6.5800	8.1353	5.3239	Inf
5	Inf	3.9120	8.8392	9.7656	9.5225	10.6467	13.1290	4.6471	5.0209	10.5099	4.8378	Inf	Inf
6	Inf	Inf	4.2351	4.8796	5.0871	7.8934	17.3682	10.0083	0.3537	9.4228	9.7466	Inf	Inf
7	Inf	Inf	4.6291	11.6860	4.6043	13.7009	9.4862	0.3298	9.3747	12.5539	0.4998	Inf	Inf
8	Inf	Inf	0.0250	6.8000	0.1689	10.9477	13.7255	5.6910	4.7075	11.4668	5.4085	Inf	Inf
9	Inf	4.0684	17.6312	11.1468	19.3093	5.3823	3.8387	10.9679	13.0037	3.8911	12.0778	Inf	Inf
10	Inf	Inf	10.6271	3.8608	12.4739	0.2290	9.0483	13.9291	7.8705	0.4040	14.5866	Inf	Inf
11	Inf	Inf	15.8211	15.4672	16.7911	10.8365	0.3114	8.7757	16.5432	8.3351	10.1398	Inf	Inf
12	Inf	Inf	8.8170	8.1812	9.9557	5.6833	5.5210	11.7369	11.4100	4.8480	12.6485	Inf	Inf
13	Inf	0.0008	10.6428	4.9548	11.9645	7.4875	8.9867	6.4787	9.1445	6.3779	6.9510	Inf	Inf
14	Inf	Inf	6.0387	0.0688	7.5291	4.7342	13.2259	11.8399	4.4773	5.2908	11.8598	Inf	Inf
15	Inf	Inf	8.8327	9.2752	9.4463	12.9417	5.4594	4.2865	12.6840	10.8219	5.0130	Inf	Inf
16	Inf	Inf	4.2286	4.3892	5.0109	10.1885	9.6987	9.6477	8.0168	9.7348	9.9217	Inf	Inf

formState

	1	2	3	4	5	6	7	8	9	10	11	12
1	1	1	4	2	4	4	3	4	2	2	3	2
2	0	5	8	6	8	8	7	7	6	6	7	0
3	0	9	12	10	12	10	11	12	10	10	11	0
4	0	13	16	14	16	14	15	15	14	14	15	0
5	1	1	4	2	4	4	3	3	2	2	0	0
6	0	5	8	6	8	7	7	7	6	6	0	0
7	0	9	12	10	12	10	11	11	10	10	0	0
8	0	13	16	14	16	13	15	15	14	14	0	0
9	1	1	4	2	4	2	3	4	2	2	0	0
10	0	5	8	6	8	6	7	7	6	6	0	0
11	0	9	12	10	12	10	9	10	10	10	0	0
12	0	13	16	14	16	14	13	13	14	14	0	0
13	1	1	4	2	4	2	3	3	2	2	0	0
14	0	5	8	6	8	5	7	7	6	6	0	0
15	0	9	12	10	12	10	9	9	10	10	0	0
16	0	13	16	14	16	13	13	13	14	14	0	0

usedSym

	1	2	3	4	5	6	7	8	9	10	11	12
1	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0
5	1	1	1	1	1	1	1	1	1	1	0	0
6	0	1	1	1	1	1	1	1	1	1	0	0
7	0	1	1	1	1	1	1	1	1	1	0	0
8	0	1	1	1	1	1	1	1	1	1	0	0
9	2	2	2	2	2	2	2	2	2	2	0	0
10	0	2	2	2	2	2	2	2	2	2	0	0
11	0	2	2	2	2	2	2	2	2	2	0	0
12	0	2	2	2	2	2	2	2	2	2	0	0
13	3	3	3	3	3	3	3	3	3	3	0	0
14	0	3	3	3	3	3	3	3	3	3	0	0
15	0	3	3	3	3	3	3	3	3	3	0	0
16	0	3	3	3	3	3	3	3	3	3	0	0

Command Window

估計出的 $x_0 \sim x_9$:

$x_0 = -1 -1j$, bits = 11

$x_1 = +1 -1j$, bits = 01

$x_2 = -1 -1j$, bits = 11

$x_3 = +1 -1j$, bits = 01

$x_4 = -1 +1j$, bits = 10

$x_5 = -1 +1j$, bits = 10

$x_6 = +1 -1j$, bits = 01

$x_7 = +1 -1j$, bits = 01

$x_8 = -1 +1j$, bits = 10

$x_9 = +1 -1j$, bits = 01

最小誤差 = 0.73459024

brute-force 最小誤差 = 0.73459024

brute-force 序列 = [3 1 3 1 2 2 1 1 2 1]

>>

為了驗證 viterbi 輸出沒錯，我把所有組合都跑過一遍（總共 4^{10} 種組合）寫了一個 10 層 for 迴圈，每一層分別代表 x_0 、 x_1 、...、 x_9 可能是 0、1、2、3，窮舉 x_0 、 x_1 、...、 x_9 的所有可能值，並確認輸出結果一樣